

Documentación Técnica: Proyecto ITGallery-test

1. Visión General

El proyecto consiste en una aplicación web para la gestión de obras de arte (CRUD). Permite visualizar, crear, editar y eliminar fichas de obras.

La arquitectura sigue el patrón MVC (Modelo-Vista-Controlador), con un enfoque moderno: el Backend actúa como una API REST que sirve datos JSON, y el Frontend (incrustado en vistas Blade) consume estos datos mediante JavaScript (Fetch API) para actualizar la interfaz sin recargar la página.

Actualización incluida en este documento:

- Nueva vista "Artworks list" en GET /ficha con filtros y paginación.
- Actualización del controlador FichaController para soportar listado + detalle.
- Actualización de routes/web.php para incluir /ficha (listado) y redirección / → /ficha.

2. Stack Tecnológico

- Backend Framework: Laravel (PHP).
- Base de Datos: MySQL.
- Frontend: Blade (HTML estructura) + CSS Nativo (Estilos) + JavaScript Vanilla (Lógica cliente).
- Comunicación: API RESTful (JSON) + rutas web (Blade).

3. Estructura de Base de Datos

La persistencia de datos se maneja a través de una única tabla principal llamada obras.

Campos:

- id - BIGINT (PK) - Identificador único de la obra
- titulo - VARCHAR - Título de la obra
- artista - VARCHAR - Nombre del artista
- anio - INT - Año de creación
- inventario - VARCHAR - Código interno (ej: ITG-18188)
- tamano - VARCHAR - Medidas de la obra
- imagen - VARCHAR - Ruta/nombre relativo del archivo en storage/app/public (carpeta imagenes/)
- created_at - TIMESTAMP - Fecha de creación
- updated_at - TIMESTAMP - Fecha de última modificación

4. Desglose de Archivos y Funcionalidad

A. Rutas (routes/)

1. routes/web.php

Función: Define la entrada principal de la aplicación para el navegador.

Lógica (actualizada):

- Captura la ruta raíz / y redirige a /ficha
- GET /ficha: Vista listado “Artworks list” (filtros + tabla + paginación)
- GET /ficha/{id}: Vista detalle de obra
- GET /artworks/gestion: Panel de gestión (listado + formulario alta)
- POST /artworks: Alta de nueva obra
- DELETE /artworks/{id}: Eliminación de obra
- GET /editar/{id}: Vista formulario de edición

Fragmento clave de routes/web.php:

```
Route::get("ficha", [FichaController::class, "list"])->name("ficha.list"); //
NUEVO: listado

Route::get("ficha/{id}", [FichaController::class, "index"])->name("ficha"); //
Detalle

Route::get("/", function () {

    return redirect()->route("ficha.list");

});
```

2. routes/api.php

Función: Define los puntos de acceso (endpoints) para que el JavaScript pueda leer y guardar datos.

Endpoints definidos:

- GET /api/obra/{id}: Devuelve todos los detalles de una obra específica
- PUT /api/obra/{id}: Actualiza una obra existente
- POST /api/obra/{id}: Actualiza una obra (alternativa para subida de archivos binarios/imágenes)

B. Controladores (app/Http/Controllers/)

3. Api/ObraController.php

Función: Es el cerebro de la API. Recibe las peticiones de routes/api.php, procesa la lógica y devuelve JSON.

4. ArtworkManageController.php

Función: Gestiona el panel de administración web (no API).

- index(): muestra el panel de gestión / galería.
- store(\$request): alta de nueva obra desde formulario web.
- destroy(\$id): eliminación de obra desde el panel.

5. FichaController.php (ACTUALIZADO)

Función: Gestiona el listado y el detalle de fichas de obra.

Métodos clave:

- list(Request \$request): listado con filtros (q, artist, inventory, year_from, year_to, has_image, per_page) y paginación.
- index(\$id): muestra el detalle de una obra.

Snippet clave de FichaController::list():

```
public function list(Request $request)
{
    $query = Obra::query();

    // Filtros: q (buscador general), artist, inventory,
    // year_from, year_to, has_image (0/1), per_page (15/25/50)

    $obras = $query->orderByDesc("id")
        ->paginate($perPage)
        ->withQueryString();

    return view("ficha_listado", compact("obras"));
}
```

C. Modelos (app/Models/)

6. Obra.php

Función: Representa la tabla obras como un objeto PHP (Eloquent ORM). Permite hacer consultas como `Obra::find(1)` sin escribir SQL manual.

Configuración Eloquent:

- `protected $table = 'obras'`
- `public $incrementing = true`
- `protected $keyType = 'int'`
- `protected $fillable = ['titulo','artista','anio','inventario','tamano','imagen']`

D. Base de Datos y Migraciones

Migraciones relevantes:

- Ajuste de id a `AUTO_INCREMENT (BIGINT UNSIGNED)`.
- Campo `imagen` nullable.

E. Vistas / Frontend (resources/views/)

9. artworks/gestion.blade.php

Función: Es la interfaz principal de gestión / galería.

10. editar.blade.php

Función: Interfaz de edición avanzada de una obra; envío de datos por Fetch a `/api/obra/{id}`.

11. ficha.blade.php

Función: Ficha detalle con pestañas (tabs), acciones, y scripts de UI.

12. ficha_listado.blade.php (NUEVO)

Función: vista “Artworks list” con filtros (GET) y tabla de resultados con acciones Ver/Editar.

Características:

- Sidebar completa (mismo patrón que otras vistas).
- Filtros: buscador general (q), artista, inventario, rango de años, presencia de imagen, ítems por página.
- Tabla con miniatura (`asset("storage/imagenes/...")`), campos principales y botones Ver/Editar.
- Paginación nativa de Laravel con persistencia de filtros (`withQueryString()`).
- Scripts: dropdowns de sidebar, fecha dinámica en header.

Archivo: `resources/views/ficha_listado.blade.php` (layout responsive, CSS inline).

5. Diccionario de la API (Endpoints)

- GET /api/obra/{id}: Obtener detalle obra
- PUT /api/obra/{id}: Editar obra
- POST /api/obra/{id}: Editar obra (alternativa para multipart/form-data)

6. Gestión de Archivos (Imágenes)

Ubicación:

- Disco lógico: public
- Carpeta: imagenes/

Convención de nombre:

- obra{id}_{timestamp}.{ext}

Enlace simbólico:

- Ejecutar php artisan storage:link
- Crea symlink: public/storage -> storage/app/public
- Permite acceso web: asset('storage/imagenes/...')

7. Comandos Utilizados para Despliegue

- composer install
- php artisan key:generate
- php artisan migrate
- php artisan storage:link
- php artisan serve
- php artisan optimize:clear (útil tras cambios de rutas/controladores)